Overview

Today's best coding agents rely on massive API models running in the cloud. What if every developer could rely on an autonomous agent equally capable offline, on consumer hardware?

Post-train an open model into a reliable agent that navigates complex codebases and drafts fixes for real software issues, accelerating developer workflows on everyday hardware.

Description

Note: there are two active competitions for the Gemma 4 Developer Agent Competition: this competition, and a paper track, where you can document your approach for this prediction competition.

Today's most capable coding agents rely on internet-connected models that require powerful commercial infrastructure. That puts advanced software engineering out of reach for many developers.

Small local models have made rapid progress, but they still struggle with complex, multi-turn software engineering. Large codebases are hard to navigate, so fixing bugs and adding features reliably is tough for models that fit on a single accelerator.

In this competition, you'll post-train Gemma 4 to build autonomous software engineering agents designed to amplify developers' talents. You'll explore new fine-tuning and reinforcement learning techniques to create agents that can independently navigate repositories and draft solutions, freeing up human engineers to focus on architecture, logic, and review.

Your solution could help put powerful coding assistants in the hands of millions more developers. Sharing how you did it makes advanced coding tools accessible to everyone.

Evaluation

You must provide a zip archive (`submission.zip`) that contains your Agent Config, comprising system prompts, custom tools, skills, and LoRA adapters. An `agent.yaml` file must be located at the root of the archive.

```
submission.zip
├── agent.yaml                    # REQUIRED: Root agent config
├── configs/
|   └── sampling.yaml             # Optional: Generation parameters
loaded via !include
├── prompts/
|   ├── system.md                 # Optional: System instructions loaded
via !include
|   └── analyzer.md
├── sub_agents/
|   └── code_analyzer.yaml        # Optional: Sub-agent or AgentTool YAML
configurations
├── adapters/                     # Optional: Fine-tuned PEFT LoRA
adapters or model weights
|   ├── main_lora/
|   |   ├── adapter_config.json
|   |   └── adapter_model.safetensors
|   └── tool_lora/
|       ├── adapter_config.json
|       └── adapter_model.safetensors
└── skills/                       # Optional: ADK Skill directories
    └── repo_navigation/
        ├── SKILL.md
        ├── scripts/              # Python or bash scripts executed in a
sandbox
        └── resources/            # Domain knowledge markdown files
```

The config language follows the Google ADK Agent Config specification with additional restrictions to prevent code execution outside of a sandbox. The submissions themselves are compiled into ADK agents to be evaluated.

Issue Scoring

Similar to SWE-Bench, your agent's submitted patches are evaluated by a PASS/FAIL metric. For each issue, the submitted patch is applied to that issue's code repository and that issue's validation tests are run. Your submission's overall score is the percentage of patched repositories that pass the validation tests.

Your agent has a limit of 12 hours to submit patches for all tasks, inclusive of sandbox setup time, but excluding time for patch validation. You may, but are not required, to set per-task time limits in your submission's `eval_config.yaml`.

Timeline

- **September 23, 2026** - Start Date.
- **November 12, 2026 (optional)** - Deadline to submit Research Paper.
- **November 25, 2026** - Entry Deadline. You must accept the competition rules before this date in order to compete.
- **November 25, 2026** - Team Merger Deadline. This is the last day participants may join or merge teams.
- **December 2, 2026** - Final Submission Deadline.

All deadlines are at 11:59 PM UTC on the corresponding day unless otherwise noted. The competition organizers reserve the right to update the contest timeline if they deem it necessary.

Prizes

Total Prizes Available: **$100,000**

The Gemma 4 Developer Agent Competition: $65,000

- **1st Place:** $37,000
- **2nd Place:** $18,000
- **3rd Place:** $10,000

Paper Submission Award (optional): $35,000

Participants are encouraged to submit original, unpublished research submissions on topics including, but not limited to:

- **Tuning & Optimization**: Parameter-efficient fine-tuning and Reinforcement Learning (RL) approaches tailored to improve SWE agent capabilities.
- **Code Comprehension**: Novel techniques for code-graph generation, parsing, and embedding to enhance deep comprehension of complex repositories.
- **Tasks & Benchmarks**: Development of new tasks, evaluation datasets, or resources targeting code structure, graphs, and structured code generation.
- **Graph Reasoning**: Methods and architectures for improving Large Language Model (LLM) reasoning over large-scale graphs.

Top submissions will be highlighted at a Google hosted event (such as a NeurIPS expo workshop) and awarded prizes. Submissions leveraging the accompanying released graph and embedding dataset are highly encouraged, but not a requirement for submission. Submissions must be made by **November 12, 2026 at 11:59PM UTC**.

To submit, sign up for the paper track.

Model Selection, Budget, and Harness Rules

Model Selection and LoRA Adapters

At present, we only support the `gemma-4-31b-it-qat-w4a16-ct` Gemma 4 model variant. You *must* choose this model for every agent and subagent.

You may, but are not required, to include one or more `.safetensors` format LoRA adapters with your submission. While you are restricted to a single base model, you may use different adapters for each agent in your submission:

1. Place PEFT LoRA directories (containing `adapter_config.json` and `adapter_model.safetensors`) inside `adapters/<adapter_name>/`.
2. Reference `adapter: <adapter_name>` on any `LlmAgent` in `agent.yaml` or `sub_agents/*.yaml` (for example, `adapter: main_lora` on the root coder agent and `adapter: tool_lora` on a read-only analyzer `AgentTool`).

Tool, Prompt, and Skill Rules

- **`!include` Directives**: Resolves file paths relative to the directory of the file containing the tag. For instance, in `agent.yaml`, `!include prompts/system.md` loads `prompts/system.md`.
- **Sandboxing**: Path traversal outside the submission root (e.g., via `../` or symlinks) is not allowed.
- **Allowed Tools**: Your agent can only request tools provided by the competition harness or custom subagents defined via `agent_tool`.
- **Skill Structure & Sandboxing**: Each skill must be a directory containing a `SKILL.md` manifest with YAML frontmatter (`name: <skill-name>`). Scripts executed via `run_skill_script` run securely inside the competition's persistent Docker container, sharing filesystem access with `run_command` and debiting execution time against your central budget. Agents can inspect domain knowledge files using `load_skill_resource`.

The predefined tools available to your agent are:

1. `run_command(command: str) -> str` - Executes a shell command in `/bin/bash -c` inside `/workspace`.

2. `submit_patch() -> str` - Stages untracked file intents (`git add -N .`) and captures `git diff HEAD` from `/workspace`.
3. `get_status() -> str` - Returns live budget consumption and patch status.
4. `read_file(filepath: str, start_line: int | None = None, end_line: int | None = None) -> str` - Reads a file from `/workspace` with 1-indexed inclusive line slicing.
5. `edit_file(filepath: str, old_string: str, new_string: str, allow_multiple: bool = False) -> str` - Replaces `old_string` with `new_string` in an existing non-empty file inside `/workspace`.
6. `write_file(filepath: str, content: str) -> str` - Creates or overwrites a file at `/workspace/<filepath>`, automatically creating parent directories (`mkdir -p`).
7. `get_code_neighbors(node: str, edge_type: str | None = None, max_neighbors: int = 50) -> str` - Finds incoming and outgoing neighbors of a symbol (`node`) in the repository call/dependency graph.
8. `search_similar_code(query: str, k: int = 10) -> str` - Finds top-`k` graph nodes with highest cosine similarity to `query` in the pre-computed embeddings.
9. `get_code_subgraph(nodes: list[str]) -> str` - Extracts the induced subgraph (all nodes and interconnecting edges) for a list of symbols.

See the `HARNESS_README.md` file in the dataset for detailed information about the submission format and execution environment.

Upgraded Accelerators

This competition has access to Kaggle's pool of powerful new L4x4 machines! These machines offer 96GB of GPU memory enabling submissions with much larger models. See this page for more on these machines.

What you need to know:

- **Quota usage** - Due to their limited availability, notebooks with L4x4s consume GPU quota at twice the rate of the older T4x2 and P100 machines. We may increase this rate as necessary to ensure these machines are available for submission scoring.
- **Restricted Use** - L4s are only available for notebooks attached to this competition. We will build tooling to enforce this if necessary. In the meantime, attempts to circumvent this

restriction may be enforced by Kaggle moderation with consequences including team bans from the competition or account bans.

- **No Internet** - All L4 sessions must have internet disabled.